

Nhom03 — Báo cáo thực hành Lab 1: Automating Everything as Code (DevOps / CI-CD / Git)

Môn: Thực hành Lập trình An toàn & Khai thác lỗ hổng phần mềm (NT521) — UIT InSec Lab **Nhóm:** Nhom03 · **Repository:** <https://github.com/Nhan-Laptop/Nhom03>

Họ và tên	MSSV
Nguyễn Trọng Nhân	24521236
Nguyễn Trọng Nhân	24520023
Lê Việt Hoàng	24520546

Tính kiểm chứng của báo cáo này. Mọi hình ảnh trong báo cáo được sinh **tự động từ log thực thi** ([docs/evidence/make_images.py](#)), mỗi ảnh có footer ghi rõ **file nguồn và khoảng dòng**. Các log thực thi thô, prompt gửi cho AI agent và telemetry của agent đều được lưu trong repo kèm **mã băm SHA-256** để đối chiếu ([mục 6](#)).

Tóm tắt kết quả

Hạng mục	Kết quả
B.1 — Git	7 commit: staging → commit → sửa tập tin → nhánh feature (fast-forward) → xử lý xung đột (merge commit có 2 cha) → push lên GitHub
B.2.1 — làm thủ công (nhánh manual)	3 commit: threat model (Yêu cầu 3) → sửa bug ret_val (Yêu cầu 4) → RBAC theo <code>policy.py</code> (Yêu cầu 5) ⇒ 11 test OK
B.2.2 — giao cho AI agent (nhánh agent)	3 commit, agent chạy trong sandbox không được tự commit ; 2 vòng (1 lần bị review chặn merge) ⇒ 24 test OK
Phát hiện đáng chú ý	Agent tự đổi hợp đồng trả về của hàm và viết test theo đúng cái sai ⇒ 22/22 test xanh nhưng vẫn sai đặc tả; chỉ con người review mới bắt được
Bằng chứng	30 hình minh họa (27 hình sinh từ log + 3 ảnh chụp GitHub thật), log thô + prompt/telemetry của agent + mã băm SHA-256 trong docs/evidence/
Bản để nộp	docs/report/[NT521]-Lab1_...-Hoang.pdf — 38 trang A4, kèm toàn bộ hình minh họa

Mục lục

- 0. Môi trường thực hành
- 1. Phần B.1 — Quản lý phiên bản phần mềm với Git
- 2. Phần B.2 — Vòng SSDLC thu nhỏ trên chức năng `json_search()`
- 3. Checklist trả lời 6 yêu cầu của đề bài
- 4. Cấu trúc repository và cách chạy lại
- 5. Ghi chú khi công khai repository
- 6. Bằng chứng, kiểm chứng và khả năng tái lập
- 7. Kiểm chứng trực tiếp trên GitHub

0. Môi trường thực hành

Thành phần	Phiên bản / giá trị
Hệ điều hành	Linux (kernel 7.1.5+kali-amd64, nền Debian)
Git	git version 2.53.0
Python	Python 3.14.7
GitHub CLI	gh version 2.101.0
Danh tính Git	user.name = "Nhom 03", user.email = nguyentrongnhan06cm@gmail.com (cấu hình --global)

Ghi chú về layout. Để bài minh hoạ việc khởi tạo repo trong `NhomX/git-intro`, trong khi B.1.6 lại yêu cầu `cd ~/NhomXX` rồi push và B.2 đặt thư mục `unittest/` ngay trong repo. Nhóm chọn **repo root = `Nhom03/`** để khớp với cả B.1.6 và B.2; thư mục `git-intro/` vẫn được giữ làm ghi chú nội bộ. Toàn bộ nội dung các bước (init, staging, commit, branch, merge, conflict) đều được thực hiện đầy đủ như yêu cầu.

1. Phần B.1 — Quản lý phiên bản phần mềm với Git

1.1 B.1.1 — Thiết lập Git repository (Yêu cầu 1)

```
$ git config --global user.name "Nhom 03"
$ git config --global user.email nguyentrongnhan06cm@gmail.com
$ git config --list          # kiểm tra lại danh tính người dùng
$ mkdir Nhom03 && cd Nhom03
$ git init                  # khởi tạo repository cục bộ
$ ls -a                     # -> .git
$ git status
On branch master

No commits yet

nothing to commit (create/copy files and use "git add" to track)
```

Bước 1. Cấu hình thông tin người dùng và liên kết tài khoản

```
$ git config --global user.name "Nhom 03"
```

```
$ git config --global user.email nguyentrongnhan06cm@gmail.com
```

Bước 5. Khởi tạo Git repository + kiểm tra thư mục ẩn

```
$ git init
```

```
hint: Using 'master' as the name for the initial branch. This default branch name
hint: will change to "main" in Git 3.0. To configure the initial branch name
hint: to use in all of your new repositories, which will suppress this warning,
hint: call:
```

```
hint:
```

```
hint:     git config --global init.defaultBranch <name>
```

```
hint:
```

```
hint: Names commonly chosen instead of 'master' are 'main', 'trunk' and
```

```
hint: 'development'. The just-created branch can be renamed via this command:
```

```
hint:
```

```
hint:     git branch -m <name>
```

```
hint:
```

```
hint: Disable this message with "git config set advice.defaultBranchName false"
```

```
Initialized empty Git repository in /home/nhanlaptop/UIT/NT521/TH1/Nhom03/.git/
```

```
$ ls -a
```

```
.
..
.git
.pi
```

Bước 6. Xem trạng thái repository cục bộ

```
$ git status
```

```
On branch master
```

```
No commits yet
```

```
nothing to commit (create/copy files and use "git add" to track)
```

```
---
```

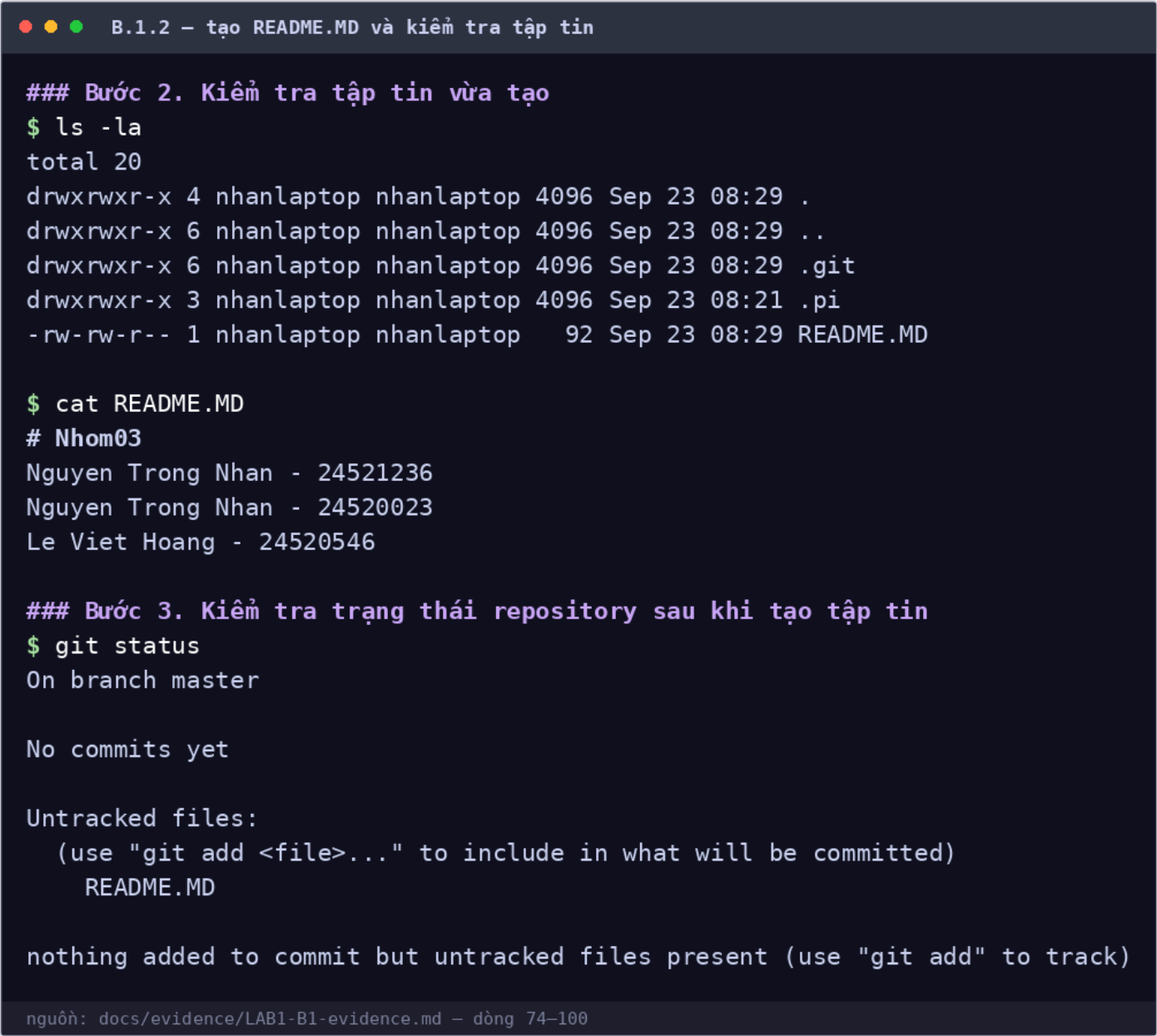
nguồn: docs/evidence/LAB1-B1-evidence.md – dòng 12–16 và 31–63

Giải thích.

- `git init` tạo thư mục ẩn `.git/` — nơi chứa toàn bộ lịch sử, đối tượng (object), tham chiếu (ref), cấu hình và reflog của repository. Đây là lý do Git không cần bất kỳ máy chủ nào vẫn quản lý được phiên bản.
- `git status` ngay sau khi init cho biết ba thông tin: **nhánh hiện tại là master**, **chưa có commit nào** (No commits yet), và **không có gì để commit** vì thư mục làm việc chưa có tập tin nào được theo dõi.
- Git 2.53 vẫn lấy `master` làm tên nhánh khởi tạo nhưng in kèm *hint* rằng Git 3.0 sẽ dùng `main`; muốn đổi mặc định thì đặt `git config --global init.defaultBranch <name>`.

1.2 B.1.2 — Staging và Committing một tập tin

```
$ cat > README.MD <<'EOF'
# Nhóm03
Nguyen Trong Nhan - 24521236
Nguyen Trong Nhan - 24520023
Le Viet Hoang - 24520546
EOF
$ ls -la && cat README.MD
```



Ba trạng thái của Git được minh chứng bằng ba lần chạy `git status` liên tiếp:

Thời điểm	git status báo	Vùng
Sau khi tạo tập tin	Untracked files: README.MD	working directory
Sau git add README.MD	Changes to be committed → new file: README.MD	staging area (index) — Git đã chụp <i>snapshot</i> của tập tin
Sau git commit	nothing to commit, working tree clean	repository (object database)

Bước 3. Kiểm tra trạng thái repository sau khi tạo tập tin

```
$ git status
```

```
On branch master
```

```
No commits yet
```

```
Untracked files:
```

```
(use "git add <file>..." to include in what will be committed)
```

```
README.MD
```

```
nothing added to commit but untracked files present (use "git add" to track)
```

Bước 4. Staging tập tin (đưa vào vùng staging)

```
$ git add README.MD
```

Bước 5. Kiểm tra trạng thái repository sau khi stage

```
$ git status
```

```
On branch master
```

```
No commits yet
```

```
Changes to be committed:
```

```
(use "git rm --cached <file>..." to unstage)
```

```
new file:   README.MD
```

Bước 6. Commit tập tin

```
$ git commit -m "Committing README.MD from Nhom03 to begin tracking changes"
```

```
[master (root-commit) 6b0c49e] Committing README.MD from Nhom03 to begin tracking changes
```

```
1 file changed, 4 insertions(+)
```

```
create mode 100644 README.MD
```

nguồn: docs/evidence/LAB1-B1-evidence.md – dòng 89–120

```
$ git commit -m "Committing README.MD from Nhom03 to begin tracking changes"
```

```
[master 6b0c49e] Committing README.MD from Nhom03 to begin tracking changes
```

```
1 file changed, 4 insertions(+)
```

```
create mode 100644 README.MD
```

Bước 6. Commit tập tin

```
$ git commit -m "Committing README.MD from Nhom03 to begin tracking changes"
[master (root-commit) 6b0c49e] Committing README.MD from Nhom03 to begin tracking changes
1 file changed, 4 insertions(+)
create mode 100644 README.MD
```

Bước 7. Xem lịch sử commit

```
$ git log
commit 6b0c49e3ee6e89d71551ac2c1557aeefd6978f08
Author: Nhom 03 <nguyentrongnhan06cm@gmail.com>
Date:   Wed Sep 23 08:29:34 2026 +0700
```

Committing README.MD from Nhom03 to begin tracking changes

nguồn: docs/evidence/LAB1-B1-evidence.md – dòng 115–130

6b0c49e là **commit ID** — 7 ký tự đầu của giá trị băm SHA-1 đầy đủ (6b0c49e3ee6e89d71551ac2c1557aeefd6978f08). Mỗi commit được định danh bằng một hash duy nhất; tùy chọn `-m` cho phép gắn message giải thích thay đổi.

1.3 B.1.3 — Sửa đổi tập tin và theo dõi các thay đổi

```
$ echo "I am beginning to understand Git" >> README.MD # ">>" nối thêm; ">" sẽ GHI ĐỀ toàn bộ nội dung cũ
$ git status # -> "Changes not staged for commit: modified: README.MD"
$ git add README.MD
$ git commit -m "Nhom03 Added additional line to file" # -> b889aa8, 1 file changed, 1 insertion(+)
```

Bước 1. Chèn thêm 1 dòng vào cuối tập tin README.MD

```
$ echo "I am beginning to understand Git" >> README.MD
```

Bước 2. Xem lại nội dung tập tin đã chỉnh sửa

```
$ cat README.MD
```

```
# Nhom03
```

```
Nguyen Trong Nhan - 24521236
```

```
Nguyen Trong Nhan - 24520023
```

```
Le Viet Hoang - 24520546
```

```
I am beginning to understand Git
```

Bước 3. Kiểm tra thay đổi đối với repository

```
$ git status
```

```
On branch master
```

```
Changes not staged for commit:
```

```
  (use "git add <file>..." to update what will be committed)
```

```
  (use "git restore <file>..." to discard changes in working directory)
```

```
    modified:   README.MD
```

```
no changes added to commit (use "git add" and/or "git commit -a")
```

Bước 4. Stage và commit tập tin đã thay đổi

```
$ git add README.MD
```

```
$ git commit -m "Nhom03 Added additional line to file"
```

```
[master b889aa8] Nhom03 Added additional line to file
```

```
1 file changed, 1 insertion(+)
```

nguồn: docs/evidence/LAB1-B1-evidence.md – dòng 133–160

```
$ git diff 6b0c49e3ee6e89d71551ac2c1557aeefd6978f08 HEAD
diff --git a/README.MD b/README.MD
index b32e865..0d2a1c2 100644
--- a/README.MD
+++ b/README.MD
@@ -2,3 +2,4 @@
 Nguyen Trong Nhan - 24521236
 Nguyen Trong Nhan - 24520023
 Le Viet Hoang - 24520546
+I am beginning to understand Git
```

B.1.3 – so sánh 2 commit bằng git diff

Bước 6. So sánh 2 commit

```
$ git diff 6b0c49e3ee6e89d71551ac2c1557aeefd6978f08 HEAD
diff --git a/README.MD b/README.MD
index b32e865..0d2a1c2 100644
--- a/README.MD
+++ b/README.MD
@@ -2,3 +2,4 @@
    Nguyen Trong Nhan - 24521236
    Nguyen Trong Nhan - 24520023
    Le Viet Hoang - 24520546
+I am beginning to understand Git
- - -
```

nguồn: docs/evidence/LAB1-B1-evidence.md – dòng 175–188

Ý nghĩa các ký hiệu trong output của git diff:

Ký hiệu	Ý nghĩa
diff --git a/... b/...	file được so sánh giữa hai commit; a/ là bản ở commit đầu, b/ là bản ở commit sau
index b32e865..0d2a1c2 100644	hash blob của nội dung cũ → mới; 100644 = file thường, không có quyền thực thi
--- / +++	dòng đánh dấu bên cũ (a/) và bên mới (b/)
@@ -2,3 +2,4 @@	<i>hunk header</i> : từ dòng 2 của bản cũ lấy 3 dòng, tương ứng từ dòng 2 của bản mới lấy 4 dòng
(dòng có dấu cách đầu)	ngữ cảnh, không thay đổi
-	dòng bị xoá khỏi bản cũ
+	dòng được thêm vào bản mới

git log cho biết mỗi commit gồm: **commit hash**, **Merge:** (chỉ có với merge commit), **Author**, **Date** và **message**.

1.4 B.1.4 — Branches và Merging

```
$ git branch feature      # tạo nhánh mới trở vào commit hiện tại
$ git branch              # master, KHÔNG có dấu * -> chưa checkout sang feature
$ git checkout feature
$ git branch              # -> * feature
```

Bước 1. Tạo branch mới tên feature

```
$ git branch feature
```

Bước 2. Kiểm tra branch hiện tại

```
$ git branch
feature
* master
```

Bước 3. Chuyển (checkout) sang branch feature

```
$ git checkout feature
Switched to branch 'feature'
```

Bước 4. Kiểm tra lại branch hiện tại

```
$ git branch
* feature
master
```

nguồn: docs/evidence/LAB1-B1-evidence.md – dòng 192–208

```
$ git checkout master
$ cat README.MD          # KHÔNG thấy dòng của feature -> thay đổi bị cô lập trong nhánh
$ git merge feature
Fast-forward
 README.MD | 1 +
$ git branch -d feature   # Deleted branch feature (was 6f0dc98).
```

Bước 1. Chuyển về branch master

```
$ git checkout master
Switched to branch 'master'
```

Bước 2. Kiểm tra README.MD ở nhánh master (feature chưa merge nên chưa có dòng mới)

```
$ cat README.MD
# Nhóm03
Nguyen Trong Nhan - 24521236
Nguyen Trong Nhan - 24520023
Le Viet Hoang - 24520546
I am beginning to understand Git
```

Bước 3. Merge branch feature vào master

```
$ git merge feature
Updating b889aa8..6f0dc98
Fast-forward
 README.MD | 1 +
 1 file changed, 1 insertion(+)
```

Bước 4. Kiểm tra nội dung README.MD sau khi merge

```
$ cat README.MD
# Nhóm03
Nguyen Trong Nhan - 24521236
Nguyen Trong Nhan - 24520023
Le Viet Hoang - 24520546
I am beginning to understand Git
This is a new line from branch feature
```

Bước 5. Xoá branch feature

```
$ git branch
  feature
* master

$ git branch -d feature
Deleted branch feature (was 6f0dc98).

$ git branch
* master
```

nguồn: docs/evidence/LAB1-B1-evidence.md – dòng 253–293

Câu trả lời cho các điểm hỏi trong đề.

- Dấu * trước tên nhánh trong `git branch` chính là nhánh đang được **checkout** (HEAD trỏ tới nó).
- Sau khi merge, nội dung README.MD trên master **bằng đúng** nội dung cuối của nhánh feature — không mất dòng nào, chỉ thêm dòng mới.
- Lần merge này là **Fast-forward**: vì master không có commit nào phát sinh kể từ lúc tách nhánh, Git chỉ cần **dời con trỏ** master tới commit của feature, do đó **không có merge commit** và lịch sử vẫn là một đường thẳng.
- Không thể xoá nhánh đang đứng trên đó, nên phải `git checkout master` trước khi `git branch -d feature`. `-d` chỉ xoá khi nhánh đã được merge (an toàn); `-D` xoá cưỡng bức.

1.5 B.1.5 — Xử lý xung đột khi merge

```
$ git branch test && git checkout test
$ sed -i 's/feature/test/' README.MD          # -> "... from branch test"
$ git commit -a -m "branch test Change feature to test"    # eda981f
$ git checkout master
$ sed -i 's/feature/master/' README.MD          # -> "... from branch master"
$ git commit -a -m "branch master Changed feature to master" # 62a858e
```

Bước 2. Chỉnh sửa nội dung README.MD bằng sed

```
$ sed -i 's/feature/test/' README.MD
```

Nội dung README.MD TRƯỚC khi chạy:

```
$ cat README.MD
```

```
# Nhom03
```

```
Nguyen Trong Nhan - 24521236
```

```
Nguyen Trong Nhan - 24520023
```

```
Le Viet Hoang - 24520546
```

```
I am beginning to understand Git
```

```
This is a new line from branch feature
```

Nội dung README.MD SAU khi chạy (dòng cuối đổi 'feature' -> 'test'):

```
$ cat README.MD
```

```
# Nhom03
```

```
Nguyen Trong Nhan - 24521236
```

```
Nguyen Trong Nhan - 24520023
```

```
Le Viet Hoang - 24520546
```

```
I am beginning to understand Git
```

```
This is a new line from branch test
```

Bước 3. Stage + commit tập tin trên branch test

```
$ git commit -a -m "branch test Change feature to test"
```

```
[test eda981f] branch test Change feature to test
```

```
1 file changed, 1 insertion(+), 1 deletion(-)
```

Bước 4. Chuyển sang branch master và chỉnh sửa README.MD

```
$ git checkout master
```

```
Switched to branch 'master'
```

```
$ sed -i 's/feature/master/' README.MD
```

```
$ cat README.MD
```

```
# Nhom03
```

```
Nguyen Trong Nhan - 24521236
```

```
Nguyen Trong Nhan - 24520023
```

```
Le Viet Hoang - 24520546
```

```
I am beginning to understand Git
```

```
This is a new line from branch master
```

Bước 5. Stage + commit tập tin trên branch master

```
$ git commit -a -m "branch master Changed feature to master"
```

```
[master 62a858e] branch master Changed feature to master
```

```
1 file changed, 1 insertion(+), 1 deletion(-)
```

```
$ git merge test
Auto-merging README.MD
CONFLICT (content): Merge conflict in README.MD
Automatic merge failed; fix conflicts and then commit the result.
$ git status
On branch master
You have unmerged paths.
...
Unmerged paths:
  both modified:   README.MD
```

B.1.5 – merge gây xung đột: CONFLICT + unmerged paths

Bước 6. Merge hai branch test và master

```
$ git merge test
Auto-merging README.MD
CONFLICT (content): Merge conflict in README.MD
Automatic merge failed; fix conflicts and then commit the result.
```

-> exit status = 1 (khác 0 => merge KHÔNG tự động được vì xung đột)

Bước 7. Xem các commit (HEAD đang ở branch master)

```
$ git log --decorate --oneline --graph --all
* 62a858e (HEAD -> master) branch master Changed feature to master
| * eda981f (test) branch test Change feature to test
|/
* 6f0dc98 Added a third line in feature branch
* b889aa8 Nhom03 Added additional line to file
* 6b0c49e Committing README.MD from Nhom03 to begin tracking changes
```

```
$ git status
On branch master
You have unmerged paths.
  (fix conflicts and run "git commit")
  (use "git merge --abort" to abort the merge)
```

```
Unmerged paths:
  (use "git add <file>..." to mark resolution)
    both modified:   README.MD
```

```
no changes added to commit (use "git add" and/or "git commit -a")
```

Giải thích lệnh `sed -i 's/feature/test/' README.MD`: `sed` biên tập luồng văn bản; `-i` (*in-place*) ghi thẳng kết quả ngược lại tập tin (không cần `> file`); biểu thức `s/<x>/<y>/` **thay thế lần xuất hiện đầu tiên** trên **mỗi dòng** (thêm `g` để thay tất cả, thêm `I` để không phân biệt hoa/thường). Tùy chọn `-a` của `git commit` tự động stage các tập tin **đã bị sửa hoặc xóa**, nhưng **không** bao gồm tập tin mới chưa từng được theo dõi.

Vì sao xung đột? Hai nhánh cùng sửa **đúng một dòng** (dòng cuối) xuất phát từ cùng một *merge base* là `6f0dc98`. Git tự hoà được những vùng sửa khác nhau, nhưng ở cùng một vị trí mà có hai phiên bản khác nhau thì không thể tự quyết định — nên nó đánh dấu và dừng lại.

Đánh dấu xung đột trong tập tin:

```
<<<<<< HEAD                                # phân của nhánh đang đứng (master)
This is a new line from branch master
=====                                     # vạch phân cách
This is a new line from branch test
>>>>>> test                                # phân của nhánh được merge
```

Cách giải quyết: mở `README.MD`, giữ lại một trong hai dòng, **xoá hết ba dòng marker**, sau đó `git add README.MD` để báo cho Git biết xung đột đã được giải quyết, rồi commit:

```
B.1.5 – xoá conflict marker và kiểm tra kết quả

### Bước 8. Tìm và xoá xung đột trong README.MD
README.MD chứa conflict marker:
$ cat README.MD
# Nhom03
Nguyen Trong Nhan - 24521236
Nguyen Trong Nhan - 24520023
Le Viet Hoang - 24520546
I am beginning to understand Git
<<<<<< HEAD
This is a new line from branch master
=====
This is a new line from branch test
>>>>>> test

Xoá 3 dòng marker và dòng của branch test (giữ lại dòng của branch master - HEAD):
$ sed -i -e '/^<<<<<</d' -e '/^=====$/d' -e '/^>>>>>>/d' -e '/from branch test$/d' README.MD
Kiểm tra kết quả sau khi xoá xung đột:
$ cat README.MD
# Nhom03
Nguyen Trong Nhan - 24521236
Nguyen Trong Nhan - 24520023
Le Viet Hoang - 24520546
I am beginning to understand Git
This is a new line from branch master

nguồn: docs/evidence/LAB1-B1-evidence.md – dòng 373–397
```

```
$ git add README.MD
$ git commit -a -m "Manually merged from test branch"
[master a6c11b2] Manually merged from test branch
$ git log -1
commit a6c11b2...
Merge: 62a858e eda981f      # <- merge commit có HAI cha
```

Bước 9. Stage, commit và kiểm tra log của branch master

```
$ git add README.MD
```

```
$ git commit -a -m "Manually merged from test branch"
[master a6c11b2] Manually merged from test branch
```

```
$ git log
commit a6c11b2bf5d5eeb6a812d690a24be37eb082bffe
Merge: 62a858e eda981f
Author: Nhom 03 <nguyentrongnhan06cm@gmail.com>
Date:   Wed Sep 23 08:30:07 2026 +0700
```

Manually merged from test branch

```
commit 62a858e45e2d1e686bbcacb87aaf242d7cb59f3a
Author: Nhom 03 <nguyentrongnhan06cm@gmail.com>
Date:   Wed Sep 23 08:30:07 2026 +0700
```

branch master Changed feature to master

```
commit eda981f4b1548f59e19d34f1844625f3033ce42d
Author: Nhom 03 <nguyentrongnhan06cm@gmail.com>
Date:   Wed Sep 23 08:30:07 2026 +0700
```

branch test Change feature to test

```
commit 6f0dc98809d00cd4908f0fbd873e21b8ff1b3b28
Author: Nhom 03 <nguyentrongnhan06cm@gmail.com>
Date:   Wed Sep 23 08:29:39 2026 +0700
```

Added a third line in feature branch

nguồn: docs/evidence/LAB1-B1-evidence.md — dòng 398–428

`git cat-file -p HEAD` xác nhận commit kết quả có hai dòng parent (62a858e45e2d... và eda981f4b154...) — dấu hiệu của một **merge commit thật sự**, khác với fast-forward ở B.1.4.

1.6 B.1.6 — Tích hợp Git với GitHub (Yêu cầu 2) — Cách A: GitHub CLI

GitHub **không còn cho push bằng mật khẩu tài khoản**, nên phải xác thực bằng token hoặc khoá SSH. Nhóm chọn **Cách A**:

```
$ gh auth login --hostname github.com --git-protocol https --web
! One-time code (F99F-1AF7) copied to clipboard
Open this URL to continue in your web browser: https://github.com/login/device
$ gh auth setup-git      # để Git tự dùng token của gh làm credential helper
$ gh auth status
github.com
✓ Logged in to github.com account Nhan-Laptop (keyring)
- Git operations protocol: https
- Token: gh_*****
- Token scopes: 'gist', 'read:org', 'repo', 'workflow'
```

```
B.1.6 - xác thực GitHub bằng gh CLI (Cách A)

### Bước 1. Tạo Repository riêng trên GitHub
(repo đã được tạo sẵn trên GitHub; kiểm tra lại thông tin repo)
$ gh repo view Nhan-Laptop/Nhom03 --json name,visibility,defaultBranchRef,url
{"defaultBranchRef":{"name":"master"},"name":"Nhom03","url":"https://github.com/Nhan-Laptop/Nhom03","visibility":"PUBLIC"}

### Bước 2. Xác thực tài khoản GitHub trên máy — Cách A: GitHub CLI
$ gh auth login --hostname github.com --git-protocol https --web

! One-time code (F99F-1AF7) copied to clipboard
Open this URL to continue in your web browser: https://github.com/login/device
Ghi chú: lệnh `gh auth login --web` ở trên in ra 8 ký tự one-time code và URL https://github.com/login/device rồi dừng lại chờ xác nhận
trên trình duyệt. Do tài khoản đã đăng nhập sẵn (`gh auth status` bên dưới xác nhận phiên còn hiệu lực), luồng xác thực này không cần
hoàn tất lại.

$ gh auth setup-git

$ gh auth status
github.com
✓ Logged in to github.com account Nhan-Laptop (keyring)
- Active account: true
- Git operations protocol: https
- Token: gh_*****
- Token scopes: 'gist', 'read:org', 'repo', 'workflow'

### Bước 3. Đứng đúng repository và nhánh master
(trong máy ảo của bài lab đường dẫn là ~/Nhom03, ở đây repo nằm trong thư mục bài tập)
$ pwd
/home/nhanlaptop/UIT/NT521/TH1/Nhom03

$ git checkout master
Already on 'master'

$ git log --oneline
777d732 Nhom03 Add .gitignore and group intro note
a6c11b2 Manually merged from test branch
62a858e branch master Changed feature to master
eda981f branch test Change feature to test
6f0dc98 Added a third line in feature branch
b889aa8 Nhom03 Added additional line to file
6b0c49e Committing README.MD from Nhom03 to begin tracking changes

nguồn: docs/evidence/LAB1-B1-evidence.md — dòng 486–524
```

gh auth setup-git ghi vào ~/.gitconfig hai dòng credential.https://github.com.helper = !/usr/bin/gh auth git-credential, nhờ đó Git hỏi token qua gh (token lưu trong keyring của hệ điều hành, không nằm trong .git/config).

```
$ cd ~/Nhom03 && git checkout master && git log --oneline
$ git remote add origin https://github.com/Nhan-Laptop/Nhom03.git
$ git remote -v
origin https://github.com/Nhan-Laptop/Nhom03.git (fetch)
origin https://github.com/Nhan-Laptop/Nhom03.git (push)
$ git push -u origin master --force
```

Bước 4. Trỏ Git repository cục bộ đến GitHub repository

```
$ git remote add origin https://github.com/Nhan-Laptop/Nhom03.git
```

```
$ git remote -v
```

```
origin      https://github.com/Nhan-Laptop/Nhom03.git (fetch)
```

```
origin      https://github.com/Nhan-Laptop/Nhom03.git (push)
```

Bước 5. Gửi toàn bộ mã nguồn và lịch sử lên GitHub

```
$ git push -u origin master --force
```

```
To https://github.com/Nhan-Laptop/Nhom03.git
```

```
+ a8afbf1...777d732 master -> master (forced update)
```

```
branch 'master' set up to track 'origin/master'.
```

```
-> exit status = 0
```

nguồn: docs/evidence/LAB1-B1-evidence.md – dòng 525–538

Lệnh push cần `--force` vì lịch sử cục bộ đã được **khởi tạo lại từ đầu** (reset sạch để báo cáo đúng trình tự từng bước của đề), nên hai phía không còn chung tổ tiên. Branch `master` trên GitHub được thay bằng chuỗi commit của nhóm.

```
B.1.6 – kiểm tra kết quả trên GitHub

### Bước 6. Kiểm tra kết quả trên GitHub
$ git ls-remote origin
777d7323c2dc3c2e87377b327381229941e14956 HEAD
777d7323c2dc3c2e87377b327381229941e14956 refs/heads/master

$ gh repo view Nhan-Laptop/Nhom03 --json name,visibility,pushedAt,url
{"name":"Nhom03","pushedAt":"2026-09-23T01:36:14Z","url":"https://github.com/Nhan-Laptop/Nhom03","visibility":"PUBLIC"}

$ gh api repos/Nhan-Laptop/Nhom03/commits --jq '.[] | "\(.sha[0:7]) \(.commit.message) "'
777d732 Nhom03 Add .gitignore and group intro note
a6c11b2 Manually merged from test branch
62a858e branch master Changed feature to master
eda981f branch test Change feature to test
6f0dc98 Added a third line in feature branch
b889aa8 Nhom03 Added additional line to file
6b0c49e Committing README.MD from Nhom03 to begin tracking changes

$ gh api repos/Nhan-Laptop/Nhom03/git/trees/master --jq '.tree[].path'
.gitignore
README.MD
git-intro

$ gh api repos/Nhan-Laptop/Nhom03/contents/README.MD --jq .content | base64 -d
# Nhom03
Nguyen Trong Nhan - 24521236
Nguyen Trong Nhan - 24520023
Le Viet Hoang - 24520546
I am beginning to understand Git
This is a new line from branch master

$ git status
On branch master
Your branch is up to date with 'origin/master'.

nothing to commit, working tree clean

nguồn: docs/evidence/LAB1-B1-evidence.md – dòng 539–574
```

1.7 Lịch sử commit của phần B.1

Commit	Nhánh	Message	Thuộc bước
6b0c49e	master	Committing README.MD from Nhom03 to begin tracking changes	B.1.2
b889aa8	master	Nhom03 Added additional line to file	B.1.3
6f0dc98	feature → master	Added a third line in feature branch	B.1.4 (fast-forward)
eda981f	test	branch test Change feature to test	B.1.5
62a858e	master	branch master Changed feature to master	B.1.5
a6c11b2	master	Manually merged from test branch (merge commit, 2 cha)	B.1.5
777d732	master	Nhom03 Add .gitignore and group intro note	B.1.6

2. Phần B.2 — Vòng SSDLC thu nhỏ trên chức năng `json_search()`

2.1 Chuẩn bị

```
$ tar -xzf ../unittest.tar.gz # recursive_json_search.py, test_data.py, test_json_search.py, policy.py
$ git add unittest/ && git commit -m "Nhom03 Add baseline unittest resources for B2" # 026c591 (master)
```

Bước 1. Đưa thư mục unittest lên branch master

```
$ git status
```

On branch master

Your branch is up to date with 'origin/master'.

Untracked files:

(use "git add <file>..." to include in what will be committed)
unittest/

nothing added to commit but untracked files present (use "git add" to track)

```
$ git add unittest/
```

```
$ git status
```

On branch master

Your branch is up to date with 'origin/master'.

Changes to be committed:

(use "git restore --staged <file>..." to unstage)
new file: unittest/policy.py
new file: unittest/recursive_json_search.py
new file: unittest/test_data.py
new file: unittest/test_json_search.py

```
$ git commit -m Nhom03 Add baseline unittest resources for B2
```

[master 026c591] Nhom03 Add baseline unittest resources for B2

4 files changed, 132 insertions(+)

create mode 100644 unittest/policy.py

create mode 100644 unittest/recursive_json_search.py

create mode 100644 unittest/test_data.py

create mode 100644 unittest/test_json_search.py

```
$ git log --oneline
```

026c591 Nhom03 Add baseline unittest resources for B2

777d732 Nhom03 Add .gitignore and group intro note

a6c11b2 Manually merged from test branch

62a858e branch master Changed feature to master

eda981f branch test Change feature to test

6f0dc98 Added a third line in feature branch

b889aa8 Nhom03 Added additional line to file

6b0c49e Committing README.MD from Nhom03 to begin tracking changes

```
B.2 – tạo nhánh manual và agent, trạng thái repository

### Bước 2. Tạo 2 branch manual và agent từ branch master
$ git branch manual

$ git branch agent

$ git branch -a
  agent
  manual
* master
  test
remotes/origin/master

$ git log --oneline --decorate --graph --all
* 026c591 (HEAD -> master, manual, agent) Nhom03 Add baseline unittest resources for B2
* 777d732 (origin/master) Nhom03 Add .gitignore and group intro note
* a6c11b2 Manually merged from test branch
| \
| * eda981f (test) branch test Change feature to test
* | 62a858e branch master Changed feature to master
| /
* 6f0dc98 Added a third line in feature branch
* b889aa8 Nhom03 Added additional line to file
* 6b0c49e Committing README.MD from Nhom03 to begin tracking changes

- - -

nguồn: docs/evidence/LAB1-B2-evidence.md – dòng 70–95
```

Nhánh	Số commit riêng	Kết quả kiểm thử
master	baseline (file mã còn trống)	0 test
manual	3	11 test — OK
agent	3	24 test — OK

2.2 B.2.1 — Phát triển thử công (nhánh manual)

2.2.1 Yêu cầu 3 — Security Requirements và Threat Model

Toàn bộ phân tích nằm trong `unittest/security-requirements.md` (commit `cbde878` trên nhánh `manual`, nay có cả trên `master`). Tóm tắt đủ bốn ý (a)–(d) mà đề yêu cầu:

(a) Actor / role được phép gọi hàm và mục đích

Role	Mục đích gọi <code>json_search()</code>	Được đọc
viewer	theo dõi tình trạng sự cố (read-only)	<code>issueSummary</code>
operator	xử lý sự cố, cần địa chỉ quản trị thiết bị	<code>issueSummary</code> , <code>managementIpAddress</code>
admin	tái cấu hình thiết bị, cần thông tin xác thực	cả ba trường
role=None	không có phiên hợp lệ	không được đọc trường nào trong <code>POLICY</code>

Quyền hạn lấy từ bảng `POLICY` trong `policy.py` (allowlist theo từng trường).

(b) Asset nhạy cảm có thể xuất hiện trong dữ liệu trả về

- `apiKey` = `SNMP-COMMUNITY-STRING-7f3a9c` — **chuỗi community SNMP**, tức thông tin xác thực để quản trị thiết bị mạng; lộ nó là lộ khả năng đọc/ghi cấu hình thiết bị.
- `managementIpAddress` = `10.10.20.21` (kèm `actualServiceId` = `10.10.20.82`) — địa chỉ quản trị, phục vụ trình sát và tấn công trực tiếp.
- Thông tin định danh thiết bị: `serialNumber`, `macAddress`, `instanceUuid`, `hostname`, `platformId`.
- Thông tin vận hành nội bộ: `issueSummary`, `issueDescription`, `impactedHosts.location` (lộ topology và mức ảnh hưởng).

(c) Trust boundary đang bị bỏ qua

Ranh giới quyết định là **caller** → `json_search()`. Bản gốc **không hề biết caller là ai**: nó trả mọi giá trị tìm được cho bất kỳ lời gọi nào. Xác thực/phân quyền ở tầng trên (web API, CLI) **không thay thế được** việc kiểm tra ngay trong hàm, vì `json_search()` là điểm vào dùng chung của thư viện và có thể bị gọi trực tiếp từ module khác, từ REPL hay từ chính bộ test.

(d) Threat ánh xạ vào Information Disclosure / Elevation of Privilege (STRIDE)

#	Nhóm STRIDE	Threat	Biện pháp giảm thiểu
T1	Information Disclosure	viewer gọi <code>json_search("apiKey", data)</code> và nhận về <code>community string</code>	allowlist theo <code>POLICY</code> trước khi thêm kết quả
T2	Information Disclosure	caller bỏ luôn tham số <code>role</code> để bypass kiểm soát truy cập	fail-closed : <code>role=None</code> /role lạ không đọc được trường được bảo vệ
T3	Elevation of Privilege	<code>operator/viewer</code> tự khai <code>role="admin"</code> và được tin (client-side trust)	<code>role</code> phải do tầng xác thực server-side cấp; test từ chối role lạ
T4	Information Disclosure	viewer không hỏi <code>apiKey</code> mà hỏi key cha <code>deviceDetails</code> để lấy nguyên cây con chứa secret	lọc đệ quy (<code>_filter_value</code>) mọi trường bị hạn chế ở mọi độ sâu
T5	Tampering / DoS	JSON dị dạng hoặc lỏng quá sâu làm hàm crash/treo	luôn trả <code>list</code> , không crash; giới hạn độ sâu là khuyến nghị mở rộng
T6	Repudiation	không có vết kiểm toán khi bị từ chối	ghi log sự kiện từ chối (ngoài phạm vi lab)

Security requirements rút ra (SR-1 ... SR-5), mỗi SR được ánh xạ 1-1 tới test:

ID	Phát biểu	Threat	Test chứng minh
SR-1	Hệ thống chỉ trả về giá trị của trường X nếu <code>role</code> nằm trong danh sách được phép đọc X	T1, T3	<code>test_wrong_role_cannot_read_secret</code> , <code>test_operator_cannot_read_api_key</code> , <code>test_admin_can_read_api_key</code> , <code>test_viewer_can_read_issue_summary</code> , <code>test_management_ip_requires_operator_or_admin</code>
SR-2	Fail-closed : không cung cấp <code>role</code> thì không được đọc trường nào trong <code>POLICY</code>	T2	<code>test_no_role_cannot_read_secret</code>
SR-3	Kết quả không chứa trường bị hạn chế ở bất kỳ độ sâu nào , kể cả khi truy vấn bằng key cha	T4	<code>test_parent_key_does_not_leak_secret</code>
SR-4	Không crash, luôn trả về <code>list</code>	T5	<code>test_search_not_found</code> , <code>test_is_a_list</code>
SR-5	Tìm kiếm đệ quy phải gộp kết quả ở mọi cấp (đúng chức năng)	—	<code>test_search_found</code> , <code>test_nested_matches_are_aggregated</code>

2.2.2 Bước 2-3 — Cài đặt và chạy bản gốc

Bản mã trong để được gỡ lại nguyên trạng vào `recursive_json_search.py`, chạy:

```
$ python3 recursive_json_search.py
[]
```

B.2.1 – bản code trong đề chạy ra kết quả rỗng

```
### Bước 2. Thiết lập hàm json_search (bản trong PDF - còn bug)
$ cat unittest/recursive_json_search.py
$ cat unittest/recursive_json_search.py
from test_data import *

def json_search(key, input_object):
    ret_val = []
    if isinstance(input_object, dict): # Iterate dictionary
        for k, v in input_object.items(): # searching key in the dict
            if k == key:
                temp = {k: v}
                ret_val.append(temp)
            if isinstance(v, dict): # the value is another dict so repeat
                json_search(key, v)
            elif isinstance(v, list): # it's a list
                for item in v:
                    if not isinstance(item, (str, int)): # if dict or list repeat
                        json_search(key, item)
        else: # Iterate a list because some APIs return JSON object in a list
            for val in input_object:
                if not isinstance(val, (str, int)):
                    json_search(key, val)
    return ret_val

print(json_search("issueSummary", data))

$ git status
On branch manual
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
    modified:   unittest/recursive_json_search.py

no changes added to commit (use "git add" and/or "git commit -a")

### Bước 3. Kiểm thử chức năng – chạy chương trình
$ python3 recursive_json_search.py
[]

→ Kết quả in ra là `[]` (list RÕNG): hàm KHÔNG trả về giá trị `issueSummary` nào, dù trong `test_data.py` có 1 giá trị nằm ở
`data['enrichmentInfo']['issueDetails']['issue'][0]['issueSummary']`. Hàm hoạt động CHƯA đúng như dự định.

nguồn: docs/evidence/LAB1-B2-evidence.md – dòng 126-169
```

Kết quả **rỗng**, tức hàm **không** trả về giá trị `issueSummary` nào, mặc dù `test_data.py` có một giá trị tại `data["enrichmentInfo"]["issueDetails"]["issue"][0]["issueSummary"]` ⇒ hàm hoạt động **không đúng như dự định**.

2.2.3 Bước 4 — Unit test

```
$ python3 -m unittest -v test_json_search.py
test_search_found (...) key should be found, return list should not be empty ... FAIL
test_search_not_found (...) key should not be found, should return an empty list ... ok
test_is_a_list (...) Should return a list ... ok
Ran 3 tests in 0.002s
FAILED (failures=1)
```

```
B.2.1 – unit test: test_search_found FAIL

test_search_not_found (test_json_search.json_search_test.test_search_not_found)
key should not be found, should return an empty list ... ok

=====
FAIL: test_search_found (test_json_search.json_search_test.test_search_found)
key should be found, return list should not be empty
-----
Traceback (most recent call last):
  File "/home/nhanlaptop/UIT/NT521/TH1/Nhom03/unittest/test_json_search.py", line 12, in test_search_found
    self.assertTrue([] != json_search(key1, data))
    ~~~~~^~~~~~
AssertionError: False is not true

-----
Ran 3 tests in 0.001s

FAILED (failures=1)
[]

-> exit status = 1

nguồn: docs/evidence/LAB1-B2-evidence.md – dòng 203–223
```

2.2.4 Yêu cầu 4 — Nguyên nhân lỗi và bản vá

Test	Kết quả	Lý do
test_search_found	FAIL	issueSummary nằm ở các nhánh con ; kết quả tìm được ở nhánh con bị bỏ ⇒ lần gọi gốc trả []
test_search_not_found	PASS (giả)	key2 không tồn tại nên [] == [] vẫn đúng kể cả khi hàm sai
test_is_a_list	PASS (giả)	kiểu trả về vẫn là list

Nguyên nhân gốc: `ret_val = []` được **khởi tạo lại ở mỗi lần gọi đệ quy**, và giá trị trả về của lần gọi con **không được nhận lại** — `json_search(key, v)` được gọi rồi bỏ kết quả. Chỉ những khớp ở *cùng cấp* với lần gọi hiện tại mới được giữ.

Bản vá (3 dòng, chỉ thêm `ret_val +=`):

```
-         json_search(key, v)
+         ret_val += json_search(key, v)
...
-         json_search(key, item)
+         ret_val += json_search(key, item)
...
-         json_search(key, val)
+         ret_val += json_search(key, val)
```

Bước 5. Phân tích test case, nguyên nhân lỗi và sửa code (Yêu cầu 4)

****So sánh 2 lần chạy unittest ở Bước 4:**** `test\_search\_found` ****FAIL****, `test\_search\_not\_found` và `test\_is\_a\_list` ****PASS****.

****Nguyên nhân gốc (root cause):**** `ret\_val = []` được khởi tạo lại ở ****mỗi lần gọi đệ quy**** và giá trị trả về của lần gọi con bị ****bỏ đi**** (không được gán/gộp vào `ret\_val` của lần gọi cha). Vì vậy hàm chỉ giữ được các match nằm ở ****cùng cấp**** với lần gọi hiện tại, còn kết quả tìm thấy ở các nhánh con thì bị mất.

- `issueSummary` nằm sâu trong dữ liệu: `data` → `enrichmentInfo` → `issueDetails` → `issue[0]` → `issueSummary`.

Mọi match đều rơi vào các lần gọi con → lần gọi gốc trả về `[]` (đúng như kết quả quan sát ở Bước 3).

- `test\_search\_not\_found` PASS (kể cả khi bug) vì `key2` không tồn tại nên `[] == []` vẫn đúng.

- `test\_is\_a\_list` PASS (kể cả khi bug) vì hàm vẫn trả về đúng kiểu `list`.

****Cách sửa:**** gộp (aggregate) kết quả của mỗi lần gọi đệ quy vào `ret\_val` của lần gọi hiện tại – sửa 3 chỗ

(`json\_search(key, v)` → `ret\_val += json\_search(key, v)`): nhánh value là dict, nhánh list bên trong dict, và nhánh input_object là list.

\$ diff -u recursive_json_search.buggy.py recursive_json_search.py # bản còn bug vs bản đã sửa

Kiểm chứng sau khi sửa:

\$ python3 recursive_json_search.py
[{'issueSummary': 'Network Device 10.10.20.82 Is Unreachable From Controller'}]]

\$ python3 -m unittest -v test_json_search.py
test_is_a_list (test_json_search.json_search_test.test_is_a_list)
Should return a list ... ok
test_search_found (test_json_search.json_search_test.test_search_found)
key should be found, return list should not be empty ... ok
test_search_not_found (test_json_search.json_search_test.test_search_not_found)
key should not be found, should return an empty list ... ok

Ran 3 tests in 0.001s

OK
[{'issueSummary': 'Network Device 10.10.20.82 Is Unreachable From Controller'}]]

→ Cả 3 test case đều OK (PASS). Hàm đã gộp được kết quả tìm kiếm đệ quy.

Bước 6. Commit code đã sửa lên nhánh manual

\$ git add recursive_json_search.py test_json_search.py
\$ git commit -m "Nhóm03 Fix ret_val bug in json_search, 3 unit tests PASS"
(đang đứng trong thư mục `unittest/`)
\$ git add recursive_json_search.py test_json_search.py

\$ git commit -m "Nhóm03 Fix ret_val bug in json_search, 3 unit tests PASS"
[manual 5542c34] Nhóm03 Fix ret_val bug in json_search, 3 unit tests PASS
2 files changed, 48 insertions(+), 2 deletions(-)

\$ git log --oneline -3
5542c34 Nhóm03 Fix ret_val bug in json_search, 3 unit tests PASS
cbde878 docs: add security requirements and threat model for json_search()
026c591 Nhóm03 Add baseline unittest resources for B2

\$ git status
On branch manual
Untracked files:
 (use "git add <file>..." to include in what will be committed)
 __pycache__/

nothing added to commit but untracked files present (use "git add" to track)

nguồn: docs/evidence/LAB1-B2-evidence.md – dòng 224–283

```
$ python3 recursive_json_search.py
[{'issueSummary': 'Network Device 10.10.20.82 Is Unreachable From Controller'}]]
$ python3 -m unittest -v test_json_search.py
Ran 3 tests in 0.001s
OK
```

Commit 5542c34 trên nhánh `manual`.

2.2.5 Yêu cầu 5 — Kiểm soát truy cập theo role và security test

Hàm được đổi chữ ký thành `json_search(key, input_object, role=None)`; POLICY trong `policy.py` trở thành nguồn chân lý duy nhất cho quyết định truy cập, kèm ba lớp:

```
def _is_allowed(key, role):          # SR-1/SR-2: chỉ cho khi role nằm trong POLICY[key]
    if key not in POLICY:
        return True                # trường không bị hạn chế
    return role in POLICY[key]

def _filter_value(value, role):      # SR-3: lọc đệ quy cây con, chặn bypass qua key cha
    if isinstance(value, dict):
        return {k: _filter_value(v, role) for k, v in value.items() if _is_allowed(k, role)}
    if isinstance(value, list):
        return [_filter_value(item, role) for item in value]
    return value
```

```
B.2.1 – RBAC theo policy.py + security test (Yêu cầu 5)

### Bước 7. Bổ sung kiểm soát truy cập theo role và viết security test (Yêu cầu 5)

Đọc và phân tích policy.py:
$ cat policy.py
# policy.py
# Quy định vai trò (role) nào được phép đọc giá trị của một trường (key)
# cụ thể trong dữ liệu JSON trả về bởi json_search().
#
# Sinh viên dùng bảng này để viết security test ở Yêu cầu 6: gọi
# json_search(key, data, role=...) với một role KHÔNG nằm trong danh sách
# cho phép của key đó, và kỳ vọng kết quả trả về là rỗng.

POLICY = {
    "apiKey": ["admin"],
    "managementIpAddress": ["admin", "operator"],
    "issueSummary": ["admin", "operator", "viewer"],
}

→ POLICY là **allowlist theo từng trường**: `apiKey` chỉ `admin`; `managementIpAddress` cho `admin`, `operator`;
`issueSummary` cho cả 3 role. Hàm `json_search()` được cập nhật thành `json_search(key, input_object, role=None)`:

- **SR-1** kiểm tra `role in POLICY[key]` trước khi append kết quả;
- **SR-2** fail-closed: `role=None` không nằm trong bất kỳ allowlist nào ⇒ không đọc được trường trong POLICY;
- **SR-3** thêm hàm `_filter_value()` lọc đệ quy mọi dict/list trong cây kết quả, chặn bypass bằng key cha.

$ git diff # thay đổi code của Bước 7 so với commit ở Bước 6
diff --git a/unittest/recursive_json_search.py b/unittest/recursive_json_search.py
index 32887f2..408b1a4 100644
--- a/unittest/recursive_json_search.py
+++ b/unittest/recursive_json_search.py
@@ -1,24 +1,70 @@
+"""recursive_json_search.py
+
+
+json_search(key, input_object, role=None)
+
+ - Tìm ĐỆ QUY mọi cặp key/value có `key` trùng trong JSON object lồng nhau (dict/list).
+ - Gộp kết quả ở MỌI cấp: kết quả của các lần gọi đệ quy được cộng dồn vào ret_val
+   (fix bug: trước đây ret_val của lần gọi con bị bỏ đi).
+ - Kiểm soát truy cập theo role dựa trên POLICY trong policy.py:
+   SR-1 allowlist theo trường: chỉ trả về trường X nếu role nằm trong POLICY[X]
+   SR-2 fail-closed: role=None hoặc role không hợp lệ KHÔNG đọc được trường trong POLICY
+   SR-3 lọc đệ quy: kết quả không chứa trường bị hạn chế ở bất kỳ độ sâu nào
+     (chống bypass bằng cách truy vấn key cha, ví dụ "deviceDetails")
+
+Tham chiếu: unittest/security-requirements.md
+"""

... (xem đầy đủ trong docs/evidence/LAB1-B2-evidence.md)

nguồn: docs/evidence/LAB1-B2-evidence.md — mục 'Bước 7'
```

Kết quả kiểm chứng bằng tay (đủ ba chiều: chặn sai, cho đúng, không chặn nhầm):

```
admin + apiKey      -> [{'apiKey': 'SNMP-COMMUNITY-STRING-7f3a9c'}]
operator + apiKey    -> []
viewer + apiKey      -> []
role=None + apiKey   -> []
viewer + issueSummary -> [{'issueSummary': 'Network Device 10.10.20.82 Is Unreachable From Controller'}]
viewer + deviceDetails -> cây con ĐÃ được loại bỏ apiKey và managementIpAddress
```

Bộ test trên nhánh `manual` gồm **11 test** (4 chức năng + 7 security, mỗi method có docstring):

```
$ python3 -m unittest -v test_json_search.py
Ran 11 tests in 0.000s
OK
```

```
B.2.1 – toàn bộ unittest trên nhánh manual: 11 test OK

### Bước 8. Commit code + security test lên nhánh manual
Lưu ý: `test_search_found` được cập nhật để truyền `role="viewer"` (bắt buộc sau khi áp dụng SR-1/SR-2 fail-closed);
3 test chức năng vẫn giữ nguyên tên và docstring. Chữ "enfore" trong PDF được sửa chính tả thành "enforce".
$ git add policy.py recursive_json_search.py test_json_search.py
$ git commit -m "feat: enforce role-based access control in json_search()"
$ git add policy.py recursive_json_search.py test_json_search.py

$ git commit -m feat: enforce role-based access control in json_search()
[manual 3039abe] feat: enforce role-based access control in json_search()
 2 files changed, 127 insertions(+), 10 deletions(-)

(policy.py không bị thay đổi so với baseline, vẫn add theo đúng lệnh trong PDF)

$ git status
On branch manual
nothing to commit, working tree clean

$ git log --oneline
3039abe feat: enforce role-based access control in json_search()
5542c34 Nhom03 Fix ret_val bug in json_search, 3 unit tests PASS
cbde878 docs: add security requirements and threat model for json_search()
026c591 Nhom03 Add baseline unittest resources for B2
777d732 Nhom03 Add .gitignore and group intro note
a6c11b2 Manually merged from test branch
62a858e branch master Changed feature to master
eda981f branch test Change feature to test
6f0dc98 Added a third line in feature branch
b889aa8 Nhom03 Added additional line to file
6b0c49e Committing README.MD from Nhom03 to begin tracking changes

### Tổng kết nhánh manual
$ git log --oneline --decorate --graph --all
* 3039abe (HEAD -> manual) feat: enforce role-based access control in json_search()
* 5542c34 Nhom03 Fix ret_val bug in json_search, 3 unit tests PASS
* cbde878 docs: add security requirements and threat model for json_search()
* 026c591 (master, agent) Nhom03 Add baseline unittest resources for B2
* 777d732 (origin/master) Nhom03 Add .gitignore and group intro note
* a6c11b2 Manually merged from test branch
| \
| * eda981f (test) branch test Change feature to test
* | 62a858e branch master Changed feature to master
| /
* 6f0dc98 Added a third line in feature branch
* b889aa8 Nhom03 Added additional line to file
* 6b0c49e Committing README.MD from Nhom03 to begin tracking changes

---
```

nguồn: docs/evidence/LAB1-B2-evidence.md – dòng 546–593

Commit 3039abe. Lưu ý: `test_search_found` phải truyền `role="viewer"` vì `issueSummary` đã nằm trong `POLICY` — thay đổi duy nhất ở bộ test chức năng, được ghi chú rõ trong commit.

2.3 B.2.2 — AI coding agent (nhánh agent , Yêu cầu 6)

Cách vận hành. Đặc tả an toàn ở mục 2.2.1 được đưa sang nhánh agent (41bdbf9). Agent được chạy **tách biệt** khỏi phiên của người kiểm duyệt:

Yếu tố	Cấu hình
Công cụ / model	Pi coding agent, <code>opencode-go/deepseek-v4-flash-vision-exp</code> (vòng 2 tự failover sang <code>qwen3.8-flash</code>)
Chế độ chạy	<code>non-interactive</code> , xuất luồng sự kiện có cấu trúc (<code>--mode json -p</code>), <code>--no-approve</code> (môi trường kín, không nạp cấu hình dự án)
“Ask for approval”	cài <code>.git/hooks/pre-commit</code> chặn cứng mọi <code>git commit</code> trong lúc agent chạy ⇒ agent chỉ ghi đề xuất; người kiểm duyệt tự chạy test, đọc diff rồi mới commit
Kiểm tra vi phạm sandbox	<code>git reflog</code> sau mỗi vòng: không có commit/checkout lạ; <code>git status</code> : chỉ 2 file mã được sửa + <code>AGENT-LOG.md</code> mới, không chạm <code>policy.py/test_data.py</code>
Prompt	<code>docs/evidence/agent/prompt.txt</code> (nguyên văn)
Agent log	<code>unittest/AGENT-LOG.md</code> (9d66705) + telemetry thô <code>docs/evidence/agent/events*.jsonl</code>

Bước 2. Chạy AI coding agent trên nhánh agent (chế độ không tự commit)

****Sandbox guard:**** cài ``.git/hooks/pre-commit`` chặn mọi lệnh ``git commit`` trong thời gian agent chạy → agent chỉ được phép ghi file, không thể tự tạo commit. Người kiểm duyệt (sinh viên) tự chạy test, đọc diff rồi mới commit.

```
$ cat .git/hooks/pre-commit
```

```
#!/bin/sh
```

```
echo "BLOCKED: commits are disabled during the AI agent sandbox run." >&2
```

```
exit 1
```

****Agent được dùng:**** AI coding agent chạy headless: ``pi --mode json -p --provider opencode-go --model deepseek-v4-flash-vision-exp`` (model ``deepseek-v4-flash-vision-exp``, giao thức qua provider ``opencode-go``).

Hai phương án spawn khác đã thu trước đó: (1) ``herd_spawn`` role worker – timeout khi khởi động pane (2 lần), (2) ``bg_delegate`` – chỉ có quyền đọc, không ghi được file. Do đó dùng Pi worker headless có đầy đủ tool.

****Prompt gửi cho agent**** (nguyên văn, đã lưu tại ``agent-run/prompt.txt``):

You are an AI coding agent working inside a sandboxed working copy of a university lab repository.

WORKING DIRECTORY: `/home/nhanlaptop/UIT/NT521/TH1/Nhom03/unittest`

HARD CONSTRAINTS – a human reviewer reviews and commits, NOT you:

- Do NOT run any git command that changes repository state: no commit / add / checkout / branch / stash / reset / restore / clean / merge. Read-only git commands (status, diff, log) are allowed.
- Only create/edit files inside `/home/nhanlaptop/UIT/NT521/TH1/Nhom03/unittest/`
- Do NOT modify `test_data.py` or `policy.py`.
- Work in an "ask for approval" spirit: make the change, self-verify, then stop and report. Do not ask questions; just complete the task.

TASK – this is the exact prompt you received from the developer (record it verbatim in your log):

Please inspect all files in the `unittest/` directory and complete the following requirements:

1. Target Directory: Work inside `unittest/` with `test_data.py`, `policy.py`, and `security-requirements.md`.
2. Function Implementation (`recursive_json_search.py`):
 - Implement `json_search(key, input_object, role=None)` with recursive search logic.
 - Aggregate results from all nested dicts and lists properly without dropping data.
 - Enforce role-based access control based on `policy.py` before returning matched items.
3. Test Suite Implementation (`test_json_search.py`):
 - Create test class `json_search_test` with docstrings inside each test method.
 - Implement 3 baseline functional tests: `test_search_found`, `test_search_not_found`, `test_is_a_list`.
 - Implement at least 3 security tests validating role permissions defined in `policy.py`.
4. Self-Verification:
 - Run: `python3 -m unittest -v test_json_search.py`
 - Ensure all unit tests and security tests pass successfully.

Read `security-requirements.md` FIRST. It defines security requirements SR-1..SR-3 and a STRIDE threat model. In particular note SR-2: omitting the role (`role=None`) must NOT grant access to protected fields (fail-closed).

DELIVERABLES:

1. Overwrite the placeholders ``recursive_json_search.py`` and ``test_json_search.py`` (both currently contain only a single comment line) with your implementation.
2. Write ``AGENT-LOG.md`` in Vietnamese documenting your own session: the developer prompt above verbatim, your plan, every file you created/changed, every verification command you ran with its key output, the number of fix/rework rounds you needed before everything passed, and the final ``python3 -m unittest -v test_json_search.py`` output. Also record the model you are.
3. Your FINAL ANSWER must report: the number of test cases implemented, the number of fix/rework rounds, the exact verification command, and the final test result line.

```
Telemetry agent vòng 1 – agent tự lập kế hoạch

### [assistant_text]
I'll start by exploring the directory and reading the security requirements.

### [reasoning]
Now let me understand the requirements.

Key points:
- `json_search(key, input_object, role=None)` recursive search
- Aggregate results from all nested dicts and lists without dropping data
- Role-based access control based on policy.py
- SR-2 fail-closed: role=None → no POLICY fields returned
- SR-3: redaction recursively at any depth, including parent key queries
- SR-4: never crash, always return list
- SR-5: aggregate nested matches

... (xem telemetry đầy đủ trong file log)

nguồn: docs/evidence/agent/events.jsonl – trích các sự kiện đầu tiên
```

Điểm quan trọng: telemetry ở Hình 26 cho thấy agent **thực sự đọc** security-requirements.md và biến các security requirement thành kế hoạch làm việc — tức chất lượng đầu ra của agent phụ thuộc trực tiếp vào đặc tả do con người viết ở Bước 1.

Các chế độ vận hành của agent và sự khác nhau (câu hỏi trong đề):

Nhóm	Chế độ	Cơ chế	Dùng ở đây?
Tương tác	TUI mặc định	hiển thị từng bước, người dùng duyệt ngay trong phiên	X
	--print (--mode text)	một-shot, in trả lời cuối rồi thoát	X
	--mode json	một-shot nhưng phát sự kiện có cấu trúc (nội dung, reasoning, tool call, token/chí phí) → chạy nền + kiểm toán được	✓ cả 2 vòng
Phê duyệt	--mode rpc	agent chạy như service, IDE/host điều khiển	X
	Ask-for-approval	agent đề xuất, người duyệt rồi áp dụng/commit	✓ (hook + review)
	--approve / --no-approve	tin hay bỏ qua cấu hình/extension/prompt nằm trong dự án	✓ --no-approve
	auto-edit / full-auto	agent tự ghi file, tự commit	X
Phạm vi tool	--tools / --exclude-tools / --no-tools	allowlist/denylist công cụ ⇒ chuyển agent sang chế độ chỉ đọc (chỉ tư vấn) hoặc không được chạy lệnh	X
Phiên	lưu session (mặc định), --continue, --session <id>, --fork	agent nhớ ngữ cảnh giữa các vòng	✓ vòng 2 dùng --session
	--no-session	phiên tạm, không lưu lịch sử	X

Điểm khác biệt cốt lõi: **ai là người duyệt** (duyệt ngay trong TUI so với duyệt sau khi chạy xong nhờ hook + review), **agent có quyền ghi file hay không** (phạm vi tool), và **agent có giữ ngữ cảnh giữa các vòng hay không** (chế độ phiên).

Diễn biến hai vòng.

Vòng	Thời lượng	Tool call	Kết quả
1	3'34"	34 (0 lỗi)	22 test; 1 lần tự chạy đỏ do chính kỳ vọng test sai (fixture có 4 khớp message, test chỉ mong 3) → agent tự sửa → Ran 22 tests / OK
review	~10'	—	người kiểm duyệt phát hiện lỗi vi phạm hợp đồng trả về : json_search("apiKey", data, role="admin") trả ['SNMP-...'] thay vì [{'apiKey': 'SNMP-...'}]; test của agent assert theo đúng cái sai nên vẫn xanh ⇒ chặn merge
2	10'09"	28 (1 lỗi)	sửa 1 dòng results.append({field: _redact(value, role)}), viết lại toàn bộ assertion qua helper assertKeyValuePairs (buộc mỗi khớp là dict đúng 1 entry , key đúng bằng key cần tìm), thêm 2 test khoá hợp đồng → Ran 24 tests / OK

```

B.2.2 – review phát hiện lỗi vi phạm hợp đồng trả về

### Bước 3–4. Kiểm tra kết quả agent và phê duyệt (lần 1)

Sau khi agent chạy xong (exit code 0), người kiểm duyệt kiểm tra:

- `git branch --show-current` = `agent`; `git reflog` KHÔNG có commit/checkout lạ do agent tạo ⇒ agent tôn trọng sandbox.
- `git status`: chỉ 2 file bị sửa (`recursive_json_search.py`, `test_json_search.py`) + file mới `AGENT-LOG.md`; `policy.py`, `test_data.py`, `security-requirements.md` không bị chạm.
- Agent viết **22 test case**, tự chạy `python3 -m unittest -v test_json_search.py` ⇒ `Ran 22 tests ... OK`.
- Người kiểm duyệt tự chạy lại: `Ran 22 tests in 0.005s / OK` (tái lập được).

**Lỗi phát hiện khi review (chặn merge) – vi phạm hợp đồng trả về:**

| | Theo đặc tả trong đề bài | Kết quả agent (vòng 1) | |
|---|---|---|---|
| | `json_search("apiKey", data, role="admin")` | ` [{'apiKey': 'SNMP-COMMUNITY-STRING-7f3a9c'}]` | ` ['SNMP-COMMUNITY-STRING-7f3a9c']` |
| | `json_search("issueSummary", data, role="viewer")` | ` [{'issueSummary': 'Network Device ... Unreachable From Controller'}]` | ` ['Network Device ... Unreachable From Controller']` |

Đề bài ghi rõ hàm "trả về list các cặp key/value" và code mẫu dùng `temp = {k: v}`; agent trả về **list giá trị trần**. Bộ test của agent assert đúng theo shape sai này (`assertEqual(json_search("apiKey", data, role="admin"), [SECRET_API_KEY])`) nên 22/22 test vẫn xanh – đây là **lỗi chức năng mà test của agent không bắt được, do con người phát hiện khi review**. ⇒ Yêu cầu agent sửa (vòng rework thứ 2), đồng thời phải cập nhật test để kiểm tra đúng hợp đồng.

nguồn: docs/evidence/LAB1-B2-evidence.md – dòng 688–699
```

```

Telemetry agent vòng 2 – agent xử lý feedback của người review

### [reasoning]
The reviewer wants the return contract changed: every element must be a single-entry dict `{key: value}`.

Let me plan the changes:

1. `recursive_json_search.py`: `_search` appends `{field: _redact(value, role)}` instead of `_redact(value, role)`. But careful: what key should the single-entry dict use – the searched key (`key`) or the field name found (`field`)? The reviewer says "single-entry dict {key: value}" and "one entry per match", and the example shows `{ 'apiKey': ... }` where key == field. The reference implementation appends `temp = {k: v}` where k is the dict key encountered (which equals the searched key when matched). Since the match condition is `k == key` (with `_keys_equal`), using the found `field` is the most faithful ("appends temp = {k: v}"). But edge case: searching for `None` key – previously returned [] because no match... Actually `_keys_equal(None, None)`? In test_data there's no None key. With a non-string key that equals some field, using `field` preserves the original key type. I'll use `field` (the encountered key) – that's exactly `temp = {k: v}`. But hmm, for consistency with "keyed by the searched key", they're equal by definition of a match. I'll use `field` and document that.

Hmm, wait: careful with keys that are unhashable? Dict keys are always hashable, fine. Also `{field: ...}` requires ... (xem telemetry đây đủ trong file log)

nguồn: docs/evidence/agent/events-round2.jsonl – trích reasoning của agent ở vòng 2
```

```

B.2.2 - vòng 2: agent sửa hợp đồng + kiểm chứng 24 test OK

### Bước 5-6. Kiểm tra code diff, chạy test và commit trên nhánh agent

Người kiểm duyệt tự chạy lại test: `Ran 24 tests in 0.005s / OK`; kiểm tra shape bằng tay:
...
admin + apiKey      -> [{'apiKey': 'SNMP-COMMUNITY-STRING-7f3a9c'}]
viewer + issueSummary -> [{'issueSummary': 'Network Device 10.10.20.82 Is Unreachable From Controller'}]
viewer + apiKey      -> []
role=None + apiKey   -> []
viewer + deviceDetails: ket qua khong chua apiKey
...

$ git status --short  # chỉ 2 file code/test bị sửa + AGENT-LOG.md mới; policy.py, test_data.py, security-requirements.md không bị chạm
$ git reflog          # xác nhận agent không tạo commit/checkout nào
$ git status --short
 M unittest/recursive_json_search.py
 M unittest/test_json_search.py
?? unittest/AGENT-LOG.md

$ git reflog -3
41bdbf9 HEAD@{0}: commit: docs: bring security specification from manual to agent branch
026c591 HEAD@{1}: checkout: moving from manual to agent
3039abe HEAD@{2}: commit: feat: enforce role-based access control in json_search()

Gỡ sandbox guard và commit kết quả của agent:
$ rm .git/hooks/pre-commit
$ git add unittest/recursive_json_search.py unittest/test_json_search.py
$ git commit -m "feat: enforce role-based access control in json_search() (AI agent implementation)"
$ git add unittest/AGENT-LOG.md
$ git commit -m "docs: add B.2.2 agent log (prompt, review rounds, verification)"
$ git add recursive_json_search.py test_json_search.py

$ git commit -m feat: enforce role-based access control in json_search() (AI agent implementation)
[agent b0001f5] feat: enforce role-based access control in json_search() (AI agent implementation)
 2 files changed, 720 insertions(+), 2 deletions(-)

$ git add AGENT-LOG.md

$ git commit -m docs: add B.2.2 agent log (prompt, review rounds, verification)
[agent 9d66705] docs: add B.2.2 agent log (prompt, review rounds, verification)
 1 file changed, 522 insertions(+)
 create mode 100644 unittest/AGENT-LOG.md

$ git status
On branch agent
nothing to commit, working tree clean

nguồn: docs/evidence/LAB1-B2-evidence.md - dòng 700-745

```

Kết luận về chất lượng đầu ra của agent: đạt (và **vượt**) `manual` về độ phủ an toàn — fail-closed triệt để hơn (role là `None`/không phải chuỗi $\Rightarrow$ không đọc được **bất kỳ** trường nào), không đi vào cây con bị hạn chế, `_redact` trả **bản sao sâu** nên không làm hỏng dữ liệu nguồn, bắt `RecursionError` và so khóa an toàn trước dữ liệu thù địch — nhưng **im lặng thay đổi hợp đồng hàm** so với đặc tả, thứ mà test tự sinh không thể phát hiện.

2.4 Bước 7 — Đối sánh hai quy trình

Tiêu chí	Thủ công (manual)	AI Agent (agent)
Mức đáp ứng security requirement	SR-1 ☒ , SR-2 ☒ (chỉ trong POLICY), SR-3 ☒ , SR-4 ☒ (không crash), SR-5 ☒ . Điểm yếu: role=None vẫn đọc được trường ngoài POLICY	SR-1 ☒ , SR-2 ☒ triệt để (mọi trường khi role không hợp lệ), SR-3 ☒ + không duyệt cây con bị hạn chế + không mutate input, SR-4 ☒ (bắt RecursionError, chống __eq__ độc), SR-5 ☒ (giữ cả khớp "falsy" như []) → đáp ứng đầy đủ hơn
Số lỗi chức năng phát hiện được / còn sót	Phát hiện 1 (bug ret_val) do test ở Bước 4 bắt được; còn sót 0	Vòng 1 tự phát hiện 1 lỗi kỳ vọng test; người review phát hiện 1 lỗi hợp đồng trả về mà 22/22 test không bắt được ; còn sót 0 sau vòng 2
Số lỗi an toàn phát hiện được / còn sót	Phát hiện 1 (T4: hỏi key cha để lấy secret) ngay ở pha threat model và chặn từ thiết kế; còn sót 1 điểm yếu mức thiết kế (role=None đọc trường ngoài POLICY, đã ghi ở mục 7 của tài liệu đặc tả)	0 lỗi hổng bị phát hiện khi đối chiếu từng SR; nhưng có 1 rủi ro quy trình : đặc tả và test do cùng một agent sinh ra có thể tự khẳng định lẫn nhau (self-consistency), cần test hợp đồng độc lập
Thời gian thực hiện (ước lượng)	≈ 2'31" wall-clock cho phần code/test, nhưng người phải có mặt ở từng bước; nếu người tự làm hết (đọc đề, viết threat model, code, test) ước lượng 2-4 giờ	≈ 13'43" cho cả 2 vòng, chạy nền không cần người , chi phí ≈ 0,115 USD; cộng ~10 phút review của người
Mức độ can thiệp của con người	Can thiệp ở mọi bước; 3 commit do người tạo; 1 lần sửa tay khi thêm RBAC (test_search_found phải truyền role)	Can thiệp 2 lần: gửi review chặn merge + yêu cầu làm lại; duyệt diff và commit. Phải giám sát thêm: model bị đổi giữa hai vòng (failover)
Khả năng tái lập	Cao — mọi bước là lệnh tắt định, đặc tả nằm trong file, test deterministic ⇒ chạy lại cho kết quả giống hệt (Ran 11 tests / OK)	Thấp hơn — cùng prompt cho code/test khác nhau giữa các lần (LLM không tắt định) và đã đổi model giữa vòng 1 và vòng 2. Phần kiểm chứng vẫn tái lập được: prompt + session + telemetry được lưu, test deterministic (Ran 24 tests / OK)

Kết luận.

- Agent vượt trội về độ phủ và tốc độ:** 24 test so với 11, xử lý thêm nhiều ca biên (dữ liệu vòng, RecursionError, role không phải chuỗi, khớp rỗng, không mutate input, không duyệt cây con bị hạn chế) trong ~14 phút với ~0,12 USD.
- "Test xanh" của agent không phải bằng chứng "đúng đặc tả".** Agent tự đối hợp đồng trả về rồi viết test khớp với cái sai của mình — chỉ người đọc kỹ đặc tả mới bắt được. Cổng review của con người vì vậy **không thể bỏ qua**, và nên có **test hợp đồng độc lập** với code.
- Điểm mạnh của quy trình thủ công nằm ở pha thiết kế an toàn.** Nhờ viết threat model STRIDE trước khi code, lỗ hổng T4 được chặn ngay từ thiết kế; agent đạt điều tương tự một phần vì chính security-requirements.md (do manual viết) đã được đưa sang nhánh agent làm đầu vào.
- Khuyến nghị vận hành:** dùng agent để **mở rộng độ phủ test và gia cố**, nhưng (i) viết đặc tả/threat model trước, (ii) chạy agent trong sandbox **không có quyền commit**, (iii) bắt buộc review đối chiếu từng security requirement + contract test trước khi merge, (iv) lưu lại prompt/telemetry để suy ra được quyết định của agent khi cần truy vết.

3. Checklist trả lời 6 yêu cầu của đề bài

Yêu cầu	Nội dung	Trạng thái	Vị trí chứng minh
1	Thực hiện các bước B.1.1 → B.1.5 và trả lời câu hỏi	✓	mục 1.1 → 1.5 (Hình 1-12), log docs/evidence/LAB1-B1-evidence.md, commit 6b0c49e → a6c11b2
2	Thiết lập Git + GitHub (B.1.6)	✓	mục 1.6 (Hình 13-15), gh auth status, git remote -v, git ls-remote origin
3	Security Requirements + Threat Model (STRIDE), đủ (a)-(d), có ≥1 threat thuộc Information Disclosure / Elevation of Privilege	✓	mục 2.2.1, unittest/security-requirements.md (cbde878)
4	Phân tích nguyên nhân và sửa recursive_json_search.py, mọi testcase pass	✓	mục 2.2.4 (Hình 19-20), commit 5542c34, Ran 3 tests / OK
5	Bổ sung role theo policy.py + tối thiểu 3 security test	✓ (7 test)	mục 2.2.5 (Hình 21-22), commit 3039abe, Ran 11 tests / OK
6	Giao toàn bộ chuỗi Bước 2-8 cho AI coding agent, tự rà soát, lưu agent log	✓	mục 2.3 (Hình 23-27), unittest/AGENT-LOG.md (9d66705), docs/evidence/agent/*, Ran 24 tests / OK

4. Cấu trúc repository và cách chạy lại

```
Nhom03/
├── .gitignore          <- .pi/, *.pdf, __pycache__/, .DS_Store, *.swp
├── README.MD          <- báo cáo này
├── git-intro/Readme.md <- ghi chú nhóm (giữ lại từ B.1)
├── unittest/          <- chỉ có từ commit 026c591
│   ├── recursive_json_search.py
│   ├── test_data.py   <- fixture Cisco DNA (KHÔNG sửa)
│   ├── test_json_search.py
│   ├── policy.py      <- POLICY allowlist theo trường (KHÔNG sửa)
│   ├── security-requirements.md <- Yêu cầu 3
│   └── AGENT-LOG.md   <- chỉ có trên nhánh agent
├── docs/evidence/     <- bằng chứng thô + ảnh minh họa
│   ├── LAB1-B1-evidence.md <- log B.1.1 → B.1.6 (586 dòng)
│   ├── LAB1-B2-evidence.md <- log B.2.1, B.2.2, bảng đối sánh (834 dòng)
│   ├── SHA256SUMS       <- mã băm của các file bằng chứng
│   ├── make_images.py   <- script sinh ảnh từ log (tái lập được)
│   └── agent/           <- prompt + telemetry của AI agent
│       ├── prompt.txt
│       ├── events.jsonl      (vòng 1)
│       └── events-round2.jsonl (vòng 2)
├── report/            <- bản đề'nộp
│   ├── make_pdf.py       <- script sinh PDF từ README (tái lập được)
│   └── [NT521]-Lab1_...-Hoang.pdf <- báo cáo 38 trang A4
└── img/              <- 30 ảnh minh họa (mỗi ảnh ghi rõ nguồn dòng)
```

```
git clone https://github.com/Nhan-Laptop/Nhom03.git && cd Nhom03

# Nhánh thủ công (Yêu cầu 3, 4, 5)
git switch manual && cd unittest
python3 recursive_json_search.py
python3 -m unittest -v test_json_search.py          # Ran 11 tests ... OK

# Nhánh AI coding agent (Yêu cầu 6)
git switch agent && cd unittest
python3 -m unittest -v test_json_search.py          # Ran 24 tests ... OK
less AGENT-LOG.md                                   # prompt + 2 vòng + kết quả kiểm chứng

# So sánh hai cách làm
git diff master..manual -- unittest
git diff master..agent -- unittest
git diff manual..agent -- unittest/recursive_json_search.py

# Sinh lại toàn bộ ảnh minh họa của báo cáo từ log (cần Pillow)
cd ../../ && python3 docs/evidence/make_images.py
```

(Chạy test phải ở trong thư mục `unittest/` vì module import `test_data/policy/recursive_json_search` là file cùng thư mục — đúng theo các lệnh trong đề.)

5. Ghi chú khi công khai repository

- `test_data.py` chứa `apiKey = "SNMP-COMMUNITY-STRING-7f3a9c"` và các địa chỉ IP `10.10.x.x` — đây là dữ liệu **mẫu giả định** đi kèm bộ tài nguyên của bài lab, không phải credential hay hạ tầng thật.
- `README.MD` công khai họ tên và MSSV của các thành viên theo đúng yêu cầu định danh bài nộp; nếu muốn giới hạn, nhóm có thể chuyển repo sang **PRIVATE**.
- File tài liệu lab (`.pdf` ở **thư mục gốc**) bị loại trừ bằng `.gitignore` vì tài liệu chỉ lưu hành nội bộ; còn **báo cáo PDF do nhóm tự sinh** trong `docs/report/` thì được đưa vào repo (`.gitignore` dùng luật `/*.pdf` thay vì `*.pdf`).

6. Bằng chứng, kiểm chứng và khả năng tái lập

6.1 Ảnh minh họa không phải “vẽ lại” — chúng sinh từ log

27 ảnh trong báo cáo được sinh bằng `docs/evidence/make_images.py`:

- Nguồn nội dung:** đọc trực tiếp từ `docs/evidence/LAB1-B1-evidence.md`, `docs/evidence/LAB1-B2-evidence.md` và `telemetry docs/evidence/agent/events*.jsonl` — **không gõ lại tay**, không chỉnh sửa nội dung.
- Mỗi ảnh có footer** ghi file nguồn và khoảng dòng, ví dụ nguồn: `docs/evidence/LAB1-B1-evidence.md` – dòng 12–16 và 31–63 ⇒ người đọc có thể mở đúng file, đúng dòng để đối chiếu từng ký tự.
- Tất định:** chạy lại `python3 docs/evidence/make_images.py` sẽ sinh lại đúng bộ ảnh (cùng Pillow + cùng font DejaVu Sans Mono).

6.2 Mã băm của bằng chứng thô

File bằng chứng	SHA-256
<code>docs/evidence/LAB1-B1-evidence.md</code>	<code>79281afb50768c57316ac1e52d561c7cf21e48bf7bcd70ca3af90930e8282680</code>
<code>docs/evidence/LAB1-B2-evidence.md</code>	<code>01d90fb9a546dde24f67888c7f80c176f70d7168e3d38f0ac374ec89f61cbaae</code>
<code>docs/evidence/agent/prompt.txt</code>	<code>d979d18a7d8a5d0780349b669912b53e3ac8416fa158e79285ac5533c82e7872</code>
<code>docs/evidence/agent/events.jsonl</code> (vòng 1)	<code>b7349fb21d0a131f3c82ae9fa7f241bbb106cf58bd3a554fd807e90cfceel1a1a</code>
<code>docs/evidence/agent/events-round2.jsonl</code> (vòng 2)	<code>75e1310f89191cd19940c6bae6e65fe5d0b8a0995bb4822a99c44cf9c2030ff3</code>

Kiểm tra lại sau khi clone:

```
cd docs/evidence && sha256sum -c SHA256SUMS
# LAB1-B1-evidence.md: OK
# LAB1-B2-evidence.md: OK
# agent/prompt.txt: OK
# agent/events.jsonl: OK
# agent/events-round2.jsonl: OK
```

6.3 Đối chiếu từng khẳng định với commit

Khẳng định trong báo cáo	Commit / đường dẫn kiểm chứng
Commit đầu tiên chứa README.MD (B.1.2)	6b0c49e
Merge fast-forward ở B.1.4	6f0dc98
Merge commit có 2 cha ở B.1.5	a6c11b2 (git show --format=%P a6c11b2 → 62a858e eda981f)
Baseline unittest/ (B.2)	026c591
Yêu cầu 3 — threat model	cbde878 — unittest/security-requirements.md
Yêu cầu 4 — bản vá ret_val	5542c34 — git show --stat 5542c34
Yêu cầu 5 — RBAC + 7 security test	3039abe
Yêu cầu 6 — code do agent viết	b0001f5
Yêu cầu 6 — agent log	9d66705 — unittest/AGENT-LOG.md

Hai chi tiết có thể kiểm chứng bằng một lệnh:

```
# (1) commit a6c11b2 thật sự là merge commit (2 dòng parent)
git show --no-patch --format='%h parents: %p' a6c11b2
# a6c11b2 parents: 62a858e eda981f

# (2) nhánh agent không có commit nào do agent tự tạo
git log --oneline master..agent      # chỉ 3 commit, tất cả do người kiểm duyệt tạo
git show --stat b0001f5 | tail -4    # 2 file: recursive_json_search.py, test_json_search.py
```

6.4 Giới hạn của báo cáo (nói rõ để tránh hiểu sai)

- 1. **Bằng chứng B.2.2 là telemetry, không phải video phiên terminal:** agent chạy ở chế độ non-interactive nên không có bản ghi hình; bằng chứng gồm prompt nguyên văn, luồng sự kiện JSON (suy nghĩ, tool call, token, chi phí) và ảnh chụp màn hình GitHub.
- 2. **Model của agent đổi giữa hai vòng** (deepseek-v4-flash-vision-exp → qwen3.8-flash do provider failover). Hệ quả: “vòng 2” không hoàn toàn cùng một model với “vòng 1” — điều này đã được tính vào tiêu chí *khả năng tái lập* ở bảng đối sánh.
- 3. **Ảnh minh họa là ảnh dựng lại từ log**, không phải ảnh chụp màn hình terminal gốc. Nội dung là nguyên văn log; riêng trang GitHub (mục 7) là **ảnh chụp thật**.
- 4. **Thời gian “thủ công” (2’31”)** là thời gian thực thi các lệnh đã chuẩn bị, không bao gồm thời gian đọc đề, phân tích và thiết kế threat model.

6.5 Bản PDF để nộp (sinh tự động từ chính README này)

```
python3 docs/report/make_pdf.py      # README.MD -> HTML -> PDF (chromium headless, A4)
```

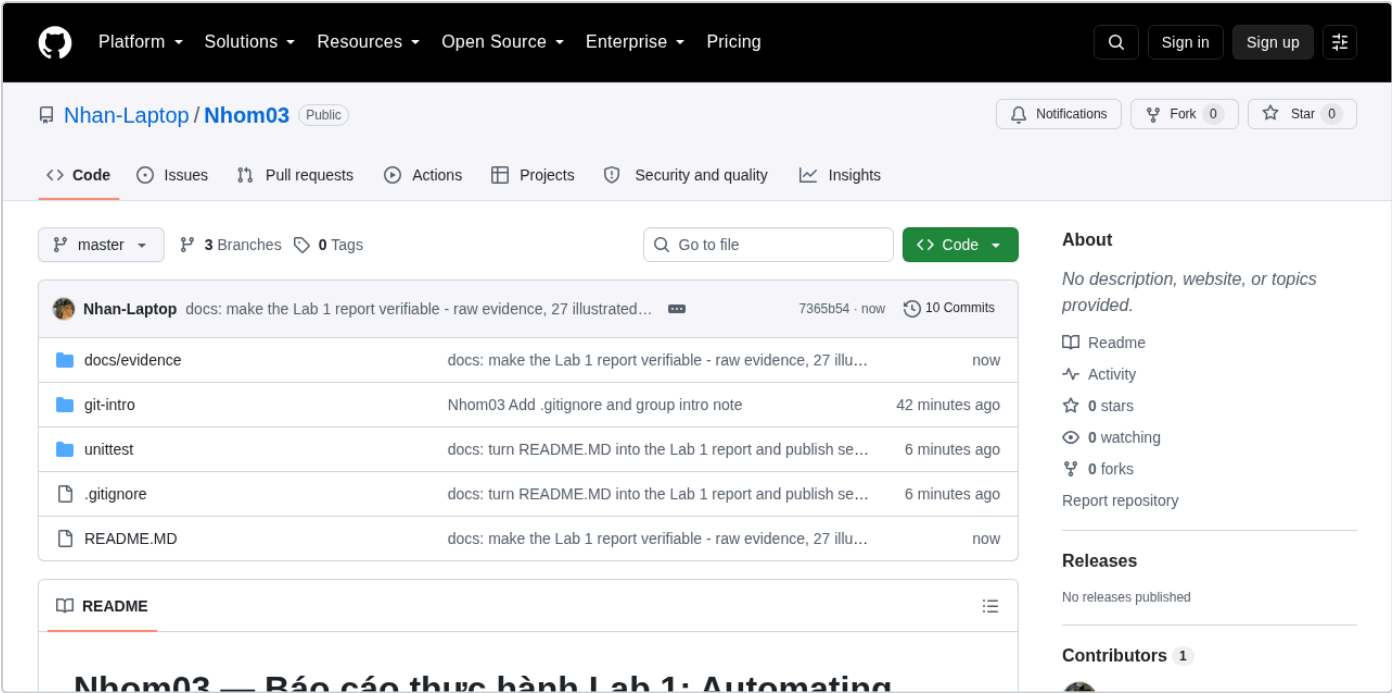
Script `docs/report/make_pdf.py` đọc README.MD, đổi đường dẫn ảnh sang đường dẫn tuyệt đối, áp CSS in A4 (tránh ngắt trang giữa bảng/ảnh) rồi in PDF bằng `chromium --headless --print-to-pdf`. Kết quả: [docs/report/\[NT521\]-Lab1_24521236-Nhan_24520023-Nhan_24520546-Hoang.pdf](#) — 38 trang A4, 3,6 MB.

Đổi tên khi nộp: theo mẫu của đề, đổi tên file thành [Mã lớp]-Lab1_MSSV1-TênSV1_MSSV2-TênSV2...pdf (ví dụ [NT521.N11.1]-Lab1_24521236-Nhan_24520023-Nhan_24520546-Hoang.pdf).

7. Kiểm chứng trực tiếp trên GitHub

(Ba ảnh dưới đây là ảnh chụp màn hình thật của repository, chụp từ trình duyệt sau khi push.)

7.1 Trang repository (nhánh mặc định `master`)



7.2 Danh sách commit — đối chiếu với bảng ở mục 1.7 và 6.3



Platform Solutions Resources Open Source Enterprise Pricing

Search

Sign in

Sign up

Filter icon

Nhan-Laptop / Nhom03Public

NotificationsFork 0Star 0

<> CodeIssuesPull requestsActionsProjectsSecurity and qualityInsights

Commits

master

All usersAll time

Commits on Sep 23, 2026

docs: make the Lab 1 report verifiable - raw evidence, 27 illustrated steps, SHA-256 checksums

Nhan-Laptop committed now

7365b54

Copy icon

<>

docs: turn README.MD into the Lab 1 report and publish security requirements (Yeu cau 3)

Nhan-Laptop committed 6 minutes ago

bffed69

Copy icon

<>

Nhom03 Add baseline unittest resources for B2

Nhan-Laptop committed 34 minutes ago

026c591

Copy icon

<>

Nhom03 Add .gitignore and group intro note

Nhan-Laptop committed 43 minutes ago

777d732

Copy icon

<>

Manually merged from test branch

Nhan-Laptop committed 45 minutes ago

a6c11b2

Copy icon

<>

branch master Changed feature to master

Nhan-Laptop committed 45 minutes ago

62a858e

Copy icon

<>

branch test Change feature to test

Nhan-Laptop committed 45 minutes ago

eda981f

Copy icon

<>

Added a third line in feature branch

Nhan-Laptop committed 45 minutes ago

6f0dc98

Copy icon

<>

Nhom03 Added additional line to file

Nhan-Laptop committed 45 minutes ago

b889aa8

Copy icon

<>

Committing README.MD from Nhom03 to begin tracking changes

Nhan-Laptop committed 45 minutes ago

6b0c49e

Copy icon

<>



© 2026 GitHub, Inc. TermsPrivacySecurityStatusCommunityDocsContactManage cookiesDo not share my personal information

7.3 Danh sách nhánh — master, manual, agent

Platform Solutions Resources Open Source Enterprise Pricing

Search Sign in Sign up

Nhan-Laptop / Nhom03Public

Notifications Fork 0 Star 0

<> Code Issues Pull requests Actions Projects Security and quality Insights

Branches

OverviewActiveStaleAll

Search branches...

Default

Branch	Updated	Check status	Behind	Ahead	Pull request
master	now			Default	

Active branches

Branch	Updated	Check status	Behind	Ahead	Pull request
agent	10 minutes ago		2	3	
manual	10 minutes ago		2	3	

© 2026 GitHub, Inc. Terms Privacy Security Status Community Docs Contact Manage cookies Do not share my personal information

Nhánh test của B.1.5 được giữ **cục bộ** (không push) vì B.1.6 chỉ yêu cầu gửi branch master; sự tồn tại và lịch sử của nó vẫn được chứng minh đầy đủ trong docs/evidence/LAB1-B1-evidence.md (Hình 9-12).